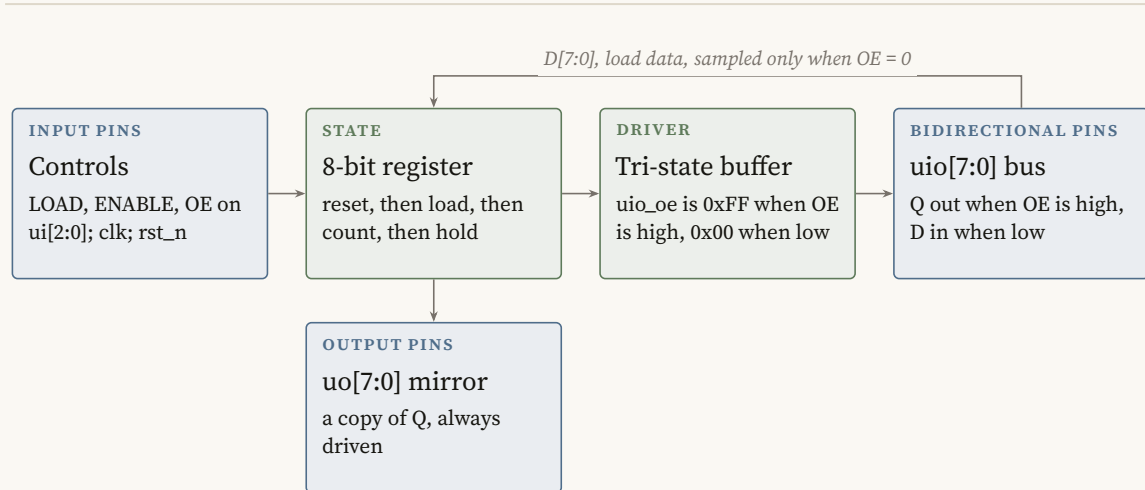


An 8-bit programmable counter on Tiny Tapeout GF180

What the design is, why its pins are laid out the way they are, how it was tested, and how it was built into a chip layout.

The assignment asked for an 8-bit programmable binary counter with an asynchronous reset, a synchronous load and tri-state outputs, tested and built for GF180. The result is one Tiny Tapeout tile whose count leaves on a bus that can go high impedance and on pins that are always driven. The load data comes in on that same bus, and that one decision shapes the rest.

FIGURE 1 *the counter as blocks*



Slate boxes are pins on the tile edge; sage boxes are logic inside it. The top arrow is the load data.

How it behaves

Four behaviours in a fixed priority. Only reset ignores the clock.

The whole counter is the single `always` block in Listing 1. Its sensitivity list names two events, the rising edge of `clk` and the falling edge of `rst_n`, and that second event is what makes the reset asynchronous: the block runs the instant `rst_n` falls, without waiting for a clock, and clears the count. Every other branch can only be reached on a clock edge, which is what makes the load and the count synchronous.

LISTING 1 *the register, from src/project.v*

```
always @(posedge clk or negedge rst_n) begin
  if (!rst_n) begin
    count <= 8'h00;          // asynchronous clear
  end else if (do_load) begin // do_load = load & ~oe
    count <= load_data;      // synchronous load, beats counting
  end else if (enable) begin
    count <= count + 8'd1;    // count up, wraps 255 -> 0
  end
  // else: hold. count keeps its value.
end
```

The order of the `if` chain is the priority, and it settles every combination of inputs:

1. *Reset.* With `rst_n` low the count is `0x00`, whatever else is happening, and it gets there between clock edges.
2. *Load.* With `LOAD` high and `OE` low, the next rising edge copies the eight bits on the bus into the register. Load is checked before enable, so raising both loads the value and does not add one to it.
3. *Count.* With `ENABLE` high and no load taking place, the next rising edge adds one. The sum is kept to eight bits, so `0xFF` plus one is `0x00` and the carry is dropped.
4. *Hold.* With neither high, nothing is assigned and the register keeps its value.

Listing 1 copied from `src/project.v`, lines 65 to 74, at commit `fd4d929`; the `do_load` comment is added.

The pin mapping

The tri-state output fills the only pins that can tri-state, so the load data has to share them.

A Tiny Tapeout tile has eight dedicated inputs (`ui`), eight dedicated outputs (`uo`) and eight bidirectional pins (`uio`). Only the bidirectional pins can be released to high impedance, through an enable byte, `uio_oe`; the dedicated outputs are always driven. A real tri-state output therefore has to live on `uio`, and an 8-bit one takes all eight of those pins.

That leaves no pins for a separate 8-bit data input. The answer is the one the 74-series bus counters use: one bus that is the output while the counter drives it and the load input while it does not. `OE` chooses which, by setting all eight bits of `uio_oe` at once.

TABLE 1 *every pin the design uses*

PIN	DIRECTION	MEANING
<code>uio[7:0]</code>	both	<code>OE</code> high: <code>uio_oe</code> = <code>0xFF</code> and the bus drives the count. <code>OE</code> low: <code>uio_oe</code> = <code>0x00</code> , the bus is high impedance and is read as the load data.
<code>ui[0]</code>	input	LOAD, synchronous parallel load, active high
<code>ui[1]</code>	input	ENABLE, count enable, active high
<code>ui[2]</code>	input	<code>OE</code> , output enable for the bus, active high
<code>ui[7:3]</code>	input	unused; tie low
<code>uo[7:0]</code>	output	a copy of the count, always driven
<code>clk</code>	input	the clock
<code>rst_n</code>	input	asynchronous reset, active low

CAN YOU LOAD WHILE THE BUS IS DRIVING?

No, and this is the price of sharing the bus. While `OE` is high the tile drives `uio`, and the GF180 pad switches its input side off (its datasheet lists input and output enabled together as disallowed), so `uio_in` has no defined value. The design therefore ignores LOAD while `OE` is high. To load, drop `OE`, put the value on the bus, then raise LOAD for one clock edge.

How it was verified

Eight CocoTB tests, a check that they catch broken designs, and the Tiny Tapeout build.

8 / 8

CocoTB tests passing in CI

4 / 4

deliberately broken counters
caught

3 / 3

Tiny Tapeout workflows passing

A test that passes against the right design proves little until it is also seen to fail against a wrong one. So the suite was run against four broken counters, each with one requirement removed: a synchronous reset in place of the asynchronous one, counting checked before loading, `uio_oe` stuck at `0xFF`, and the enable ignored. Every one failed at least one test.

The tests pass on the real counter and fail on each of four broken ones.

WHAT THE VERIFICATION ESTABLISHES

WHAT THIS DOES NOT
SHOW

The broken-counter check was run in RTL simulation only, and it covered four of the eight behaviours by mutation; the rest rely on the tests themselves. Gate-level simulation of the synthesized netlist ran in CI and passed, but nothing here has been measured on silicon.

Tests: CI run 36108189405 on fd4d929, results.xml, 8 testcases and 0 failures. Mutations: development journal, 17 September 2026.
Workflows: runs 36108189395, 36108189405 and 36108189581, 25 September 2026.

The eight tests

One requirement per test, so a failure names the requirement that broke.

The testbench in `test/` drives the tile's pins directly, the way the demo board would, and reads the count back from `uo`. Each test checks one requirement from the assignment and nothing else.

TABLE 2 *what each test checks*

TEST	PASSES ONLY IF
<code>test_async_reset</code>	<code>rst_n</code> clears the count between clock edges
<code>test_sync_load</code>	a load takes effect on the rising edge and not before it
<code>test_count_up_from_loaded_value</code>	after a load the count rises by one per edge
<code>test_count_enable_holds</code>	with <code>ENABLE</code> low the count is unchanged over many edges
<code>test_wraparound</code>	<code>0xFF</code> plus one gives <code>0x00</code>
<code>test_tristate_bus</code>	<code>OE</code> switches <code>uio_oe</code> between <code>0xFF</code> and <code>0x00</code> , and <code>uo</code> tracks the count either way
<code>test_load_beats_count</code>	with <code>LOAD</code> and <code>ENABLE</code> both high, the loaded value wins with no added one
<code>test_load_ignored_while_driving</code>	with <code>OE</code> high, <code>LOAD</code> leaves the count alone, and with <code>ENABLE</code> also high it keeps counting

Test names and checks from `test/test.py` at commit `fd4d929`; all eight passed in CI run `36108189405`.

The build

The chip built on the first try. The first run still failed, on a repository setting.

Pushing the design runs three GitHub workflows. `test` runs the CocoTB suite; `docs` checks the datasheet in `docs/info.md`; `gds` synthesizes the design, places and routes it for GF180 with LibreLane, runs the Tiny Tapeout design checks, repeats the tests against the gate-level netlist, and publishes a 3D view of the layout to GitHub Pages.

On the first push, `test` and `docs` passed, and inside `gds` the layout, the checks and the gate-level test all passed too. Only the last step failed: the Pages deploy returned *404, ensure GitHub Pages has been enabled*, because the repository's Pages source had not yet been set. Setting it to *GitHub Actions* fixed that, and a fresh `gds` run then passed every job, including the deploy. The layout viewer is live at <https://ihsan-sa.github.io/ece298a-counter/>.

One step on the way is worth knowing for next time. Re-running the failed `gds` run did not work: the re-run uploads a second copy of the Pages artifact into the same run, and the deploy refuses when it finds two with the same name. A new run, started from the Actions tab, has only one.

To rebuild it

1. Push to the repository, or open Actions, choose `gds`, and use *Run workflow*.
2. For a run that failed, start a new one rather than re-running it.
3. To run the tests on your own machine, run `make -B` in `test/`, and `make -B GATES=yes` once `gds` has produced a netlist.

Sources

- 1 Tiny Tapeout, *GF180 Verilog project template*. <https://github.com/TinyTapeout/ttgf-verilog-template>
- 2 This project's repository, *ihsan-sa/ece298a-counter*, branch `track/counter`. <https://github.com/ihsan-sa/ece298a-counter>
- 3 Tiny Tapeout, *FAQ: my GitHub action is failing on the Pages part*. <https://tinytapeout.com/faq/>
- 4 The layout in the Tiny Tapeout 3D viewer. <https://ihsan-sa.github.io/ece298a-counter/>